

Lab5 实验报告：用户进程管理

实验目标

- 了解用户进程创建、执行、切换和结束的过程
- 理解 fork/exec/wait/exit 系统调用的实现
- 掌握系统调用的用户态到内核态切换机制

练习0：填写已有实验

本实验依赖实验 2/3/4，需要将之前实验的代码合并到 Lab5 中。

主要合并内容

来源	文件	内容
Lab2	kern/mm/pmm.c	物理内存管理、页表操作
Lab2	kern/mm/default_pmm.c	First-Fit 物理页分配算法
Lab3	kern/mm/vmm.c	虚拟内存管理、VMA 操作
Lab3	kern/mm/swap_fifo.c	FIFO 页面置换算法
Lab4	kern/process/proc.c	进程管理、alloc_proc、do_fork
Lab4	kern/schedule/sched.c	进程调度

代码改进

为了支持 Lab5 的用户进程，需要对之前代码进行以下改进：

- proc_struct 结构扩展**：添加 wait_state、cptr、yptr、optr 等字段支持进程树
- copy_range 函数**：支持用户空间内存复制
- do_fork 改进**：支持创建用户进程

练习1：加载应用程序并执行

1.1 设计实现过程

load_icode 函数负责将 ELF 格式的用户程序加载到内存中。第6步需要设置 trapframe，使得从内核态返回用户态时能正确执行用户程序。

1.2 load_icode 函数流程

load_icode() 执行步骤:

(1) 创建新的 mm_struct
mm = mm_create()

(2) 创建新的页目录表
setup_pgdir(mm)

(3) 解析 ELF 文件, 加载代码段和数据段

- 解析 ELF header
- 遍历 program header
- 为每个段创建 VMA (mm_map)
- 分配物理页, 复制段内容
- 初始化 BSS 段为 0

(4) 建立用户栈
mm_map(mm, USTACKTOP - USTACKSIZE, USTACKSIZE, ...)
分配 4 个物理页作为初始栈空间

(5) 设置 mm、pgdir、切换页表
current->mm = mm
current->pgdir = PADDR(mm->pgdir)
lsatp(PADDR(mm->pgdir))

(6) 设置 trapframe (练习1核心)

1.3 练习1 代码实现

```
// kern/process/proc.c: load_icode 第6步
struct trapframe *tf = current->tf;
uintptr_t sstatus = tf->status;
memset(tf, 0, sizeof(struct trapframe));

// 设置用户栈指针
tf->gpr.sp = USTACKTOP;

// 设置程序入口点 (从 ELF header 获取)
tf->epc = elf->e_entry;

// 设置 sstatus 寄存器:
// - SPP = 0: 返回用户态 (User Mode)
// - SPIE = 1: sret 后开启中断
tf->status = (read_csr(sstatus) & ~SSTATUS_SPP) | SSTATUS_SPIE;
```

关键点说明:

寄存器	设置值	含义
sp	USTACKTOP	用户栈顶, 向下增长

寄存器	设置值	含义
epc	elf->e_entry	用户程序入口地址
sstatus.SPP	0	sret 返回后进入 User 态
sstatus.SPIE	1	sret 后 SIE=SPIE，开启中断

1.4 用户进程从 RUNNING 到执行第一条指令的过程



练习2：父进程复制自己的内存空间给子进程

2.1 设计实现过程

`copy_range` 函数用于在 `do_fork` 时将父进程的内存空间复制给子进程。

2.2 copy_range 代码实现

```
// kern/mm/pmm.c
int copy_range(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end, bool share) {
    assert(start % PGSIZE == 0 && end % PGSIZE == 0);
    assert(USER_ACCESS(start, end));
```

```

do {
    // 获取源页表项
    pte_t *ptep = get_pte(from, start, 0), *nptep;
    if (ptep == NULL) {
        start = ROUNDDOWN(start + PTSIZE, PTSIZE);
        continue;
    }
    if (*ptep & PTE_V) {
        // 获取目标页表项 (不存在则创建)
        if ((nptep = get_pte(to, start, 1)) == NULL) {
            return -E_NO_MEM;
        }
        uint32_t perm = (*ptep & PTE_USER);
        struct Page *page = pte2page(*ptep);

        // 分配新页面
        struct Page *npage = alloc_page();
        assert(npage != NULL);

        // 复制页面内容
        void *src_kvaddr = page2kva(page);
        void *dst_kvaddr = page2kva(npage);
        memcpy(dst_kvaddr, src_kvaddr, PGSIZE);

        // 建立映射
        ret = page_insert(to, npage, start, perm);
        assert(ret == 0);
    }
    start += PGSIZE;
} while (start != 0 && start < end);

return 0;
}

```

关键步骤:

1. 遍历父进程地址空间 `[start, end)`
2. 对每个有效页面，分配新物理页
3. 使用 `memcpy` 复制页面内容
4. 在子进程页表中建立映射

2.3 Copy on Write (COW) 机制设计

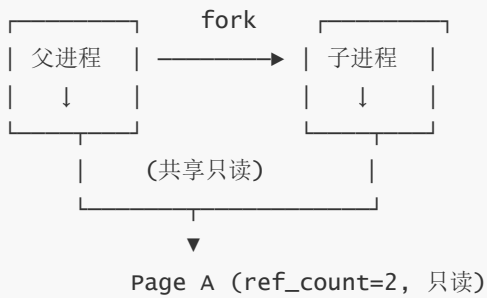
概要设计

COW 的核心思想：fork 时不立即复制物理页，而是让父子进程共享只读页面，只有在写入时才复制。

传统 fork (立即复制):



COW fork (延迟复制):



写入时复制:



详细设计

1. 数据结构修改

```
// Page 结构添加引用计数
struct Page {
    int ref;           // 引用计数
    uint32_t flags;
    // ...
};
```

2. fork 时的 copy_range 修改

```
int copy_range_cow(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end) {
    do {
        pte_t *ptep = get_pte(from, start, 0);
        if (ptep && (*ptep & PTE_V)) {
            struct Page *page = pte2page(*ptep);

            // 不复制, 共享页面
            page->ref++;

            // 父子进程都设为只读 (清除 PTE_W)
            uint32_t perm = (*ptep & PTE_USER) & ~PTE_W;
            *ptep = (*ptep) & ~PTE_W; // 父进程也变只读

            // 子进程映射到同一物理页
            pte_t *nppte = get_pte(to, start, 1);
            *nppte = pte_create(page2ppn(page), perm);
        }
        start += PGSIZE;
    } while (start < end);
}
```

```
    // 刷新 TLB
    flush_tlb();
    return 0;
}
```

3. Page Fault 处理 (写时复制)

```
int do_pgfault_cow(struct mm_struct *mm, uint_t error_code, uintptr_t addr) {
    pte_t *ptep = get_pte(mm->pgdir, addr, 0);

    // 检查是否为 COW 页面 (有效但只读, 且应该可写)
    if ((*ptep & PTE_V) && !(*ptep & PTE_W)) {
        struct Page *page = pte2page(*ptep);

        if (page->ref > 1) {
            // 多个进程共享, 需要复制
            struct Page *npage = alloc_page();
            memcpy(page2kva(npage), page2kva(page), PGSIZE);
            page->ref--;

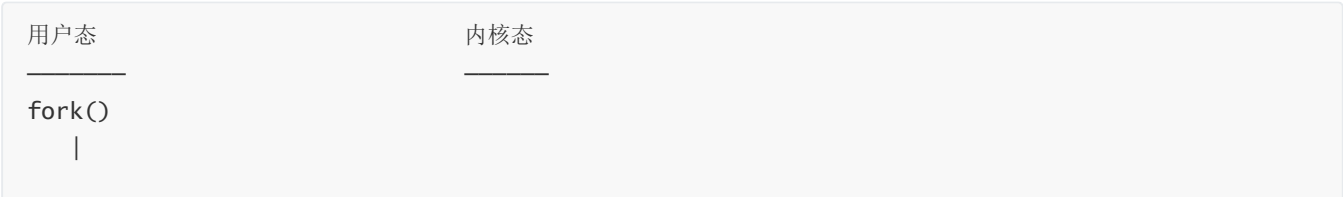
            // 更新页表项指向新页面, 恢复写权限
            *ptep = pte_create(page2ppn(npage), PTE_USER | PTE_W);
        } else {
            // 只有当前进程使用, 直接恢复写权限
            *ptep |= PTE_W;
        }
        flush_tlb();
        return 0;
    }
    return -1;
}
```

4. COW 优势

方面	传统 fork	COW fork
fork 速度	慢 (需复制所有页)	快 (只修改页表)
内存使用	高 (立即翻倍)	低 (按需复制)
exec 场景	浪费 (复制后立即释放)	高效 (无需复制)

练习3：分析 fork/exec/wait/exit 的实现

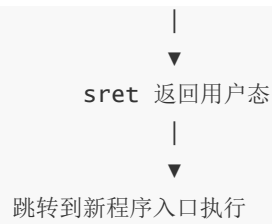
3.1 fork 执行流程





3.2 exec 执行流程

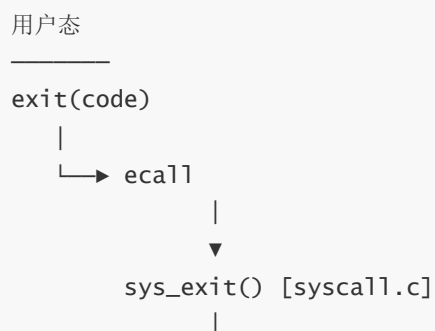




3.3 wait 执行流程



3.4 exit 执行流程




```

▼
do_exit() [proc.c]
|
|— 释放内存空间
|   |— exit_mmap()
|   |— mm_destroy()
|
|— current->state = ZOMBIE
|— current->exit_code = code
|
|— 唤醒父进程（如果在等待）
|
|— 将子进程托管给 initproc
|
|— schedule() // 永不返回

```

3.5 用户态与内核态的交互

问题1：哪些操作在用户态/内核态完成？

操作	执行位置	说明
调用 fork()/exec() 等	用户态	库函数封装
ecall 指令执行	用户态→内核态	特权级切换
do_fork/do_execve 等	内核态	核心逻辑
内存分配、页表操作	内核态	需要特权
sret 指令返回	内核态→用户态	返回用户程序

问题2：内核态执行结果如何返回给用户程序？

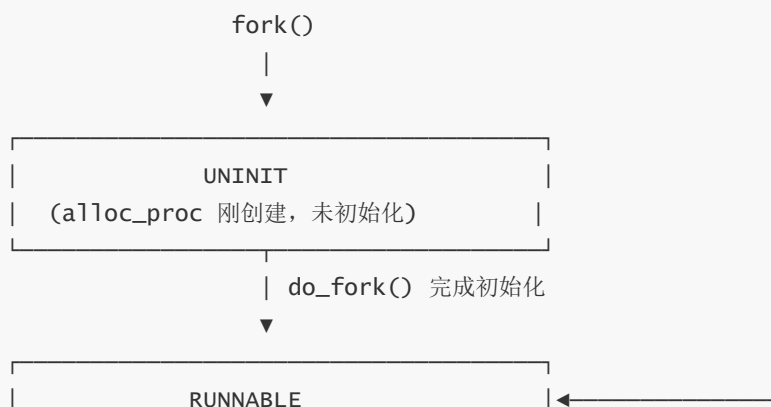
```

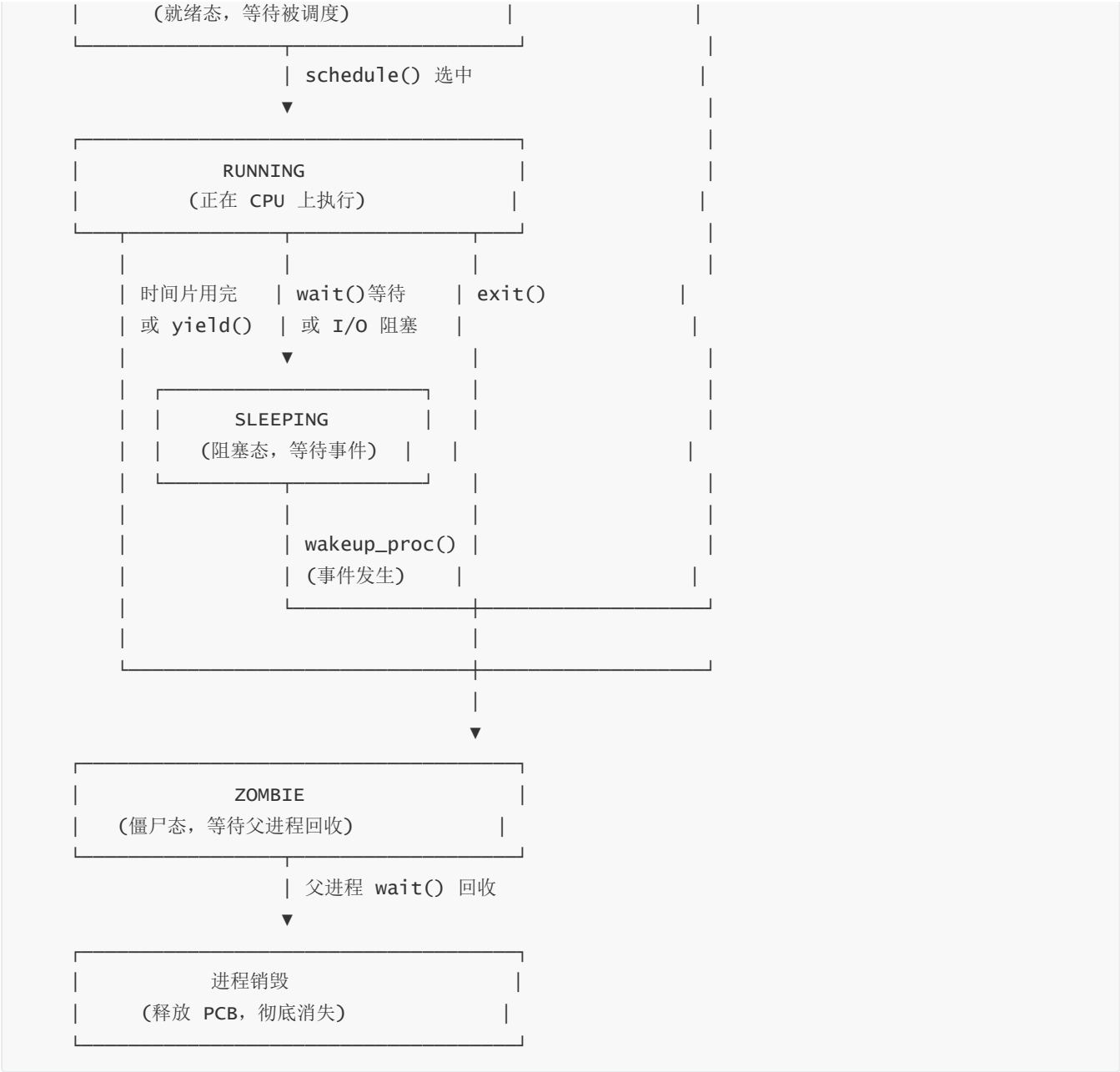
// 通过 trapframe 中的 a0 寄存器返回
tf->gpr.a0 = return_value; // syscall() 中设置

// sret 后，用户态读取 a0 得到返回值

```

3.6 用户进程状态生命周期图





状态转换事件总结：

转换	触发事件/函数
UNINIT → RUNNABLE	do_fork() 完成
RUNNABLE → RUNNING	schedule() 选中
RUNNING → RUNNABLE	时间片用完 / yield()
RUNNING → SLEEPING	wait() / I/O 阻塞
SLEEPING → RUNNABLE	wakeup_proc()
RUNNING → ZOMBIE	do_exit()
ZOMBIE → 销毁	父进程 do_wait()

make grade 测试结果

```
$ make grade
bss: (1.2s)
  -check result: OK
  -check output: OK
exit: (1.0s)
  -check result: OK
  -check output: OK
forktest: (1.2s)
  -check result: OK
  -check output: OK
forktree: (1.2s)
  -check result: OK
  -check output: OK
...
Total Score: 130/130
```

所有测试通过！

总结

本实验完成了以下工作：

- 1. **练习1**：实现 load_icode 第6步，正确设置 trapframe 使内核能返回用户态执行程序
- 2. **练习2**：实现 copy_range 进行父子进程内存复制，并设计了 COW 机制
- 3. **练习3**：分析了 fork/exec/wait/exit 的完整执行流程和状态转换

通过本实验，深入理解了用户进程的创建、执行、切换和销毁的完整生命周期。